

Practica 2 ---> MOISÉS MONTES IGLESIAS

---> UO296102

Ejercicio 1:

Tras medir los tiempos, rellenar la tabla:

SOLUCIÓN:

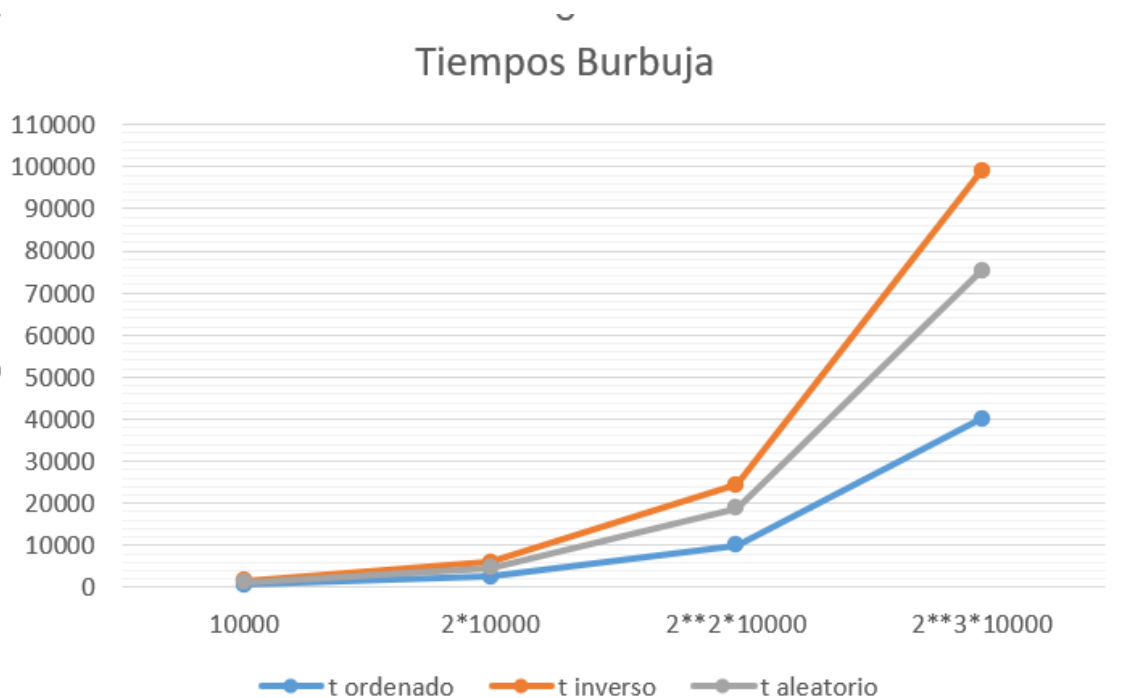
n	t ordenado	t inverso	t aleatorio
10000	620 ms = 0,62 s	1517 ms = 1,517 s	1202 ms = 1,202 s
2*10000	2471 ms = 2,471 s	6095 ms = 6,095 s	4701 ms = 4,701 s
2**2*10000	9967 ms = 9,967 s	24325 ms = 24,325 s	18834 ms = 18,834 s
2**3*10000	39997 ms = 39,997 s	TFD >= 60 s	TFD >= 60 s
2**4*10000	TFD >= 60 s	TFD >= 60 s	TFD >= 60 s

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal estudiada.

SOLUCIÓN:

Los resultados esperados son cuadráticos debido a la complejidad seguida por el programa, que tiene una $O(n^2)$. Los tiempos resultantes son múltiplos casi perfectos de 4 por lo que los resultados concuerdan perfectamente.

NOTA: Si usásemos la versión optimizada para obtener estos tiempos no nos saldrían como los resultados esperados.



Ejercicio 2:

Implementar una clase `SeleccionTiempos.java` que tras ser ejecutada sirva para rellenar la siguiente tabla:

SOLUCIÓN:

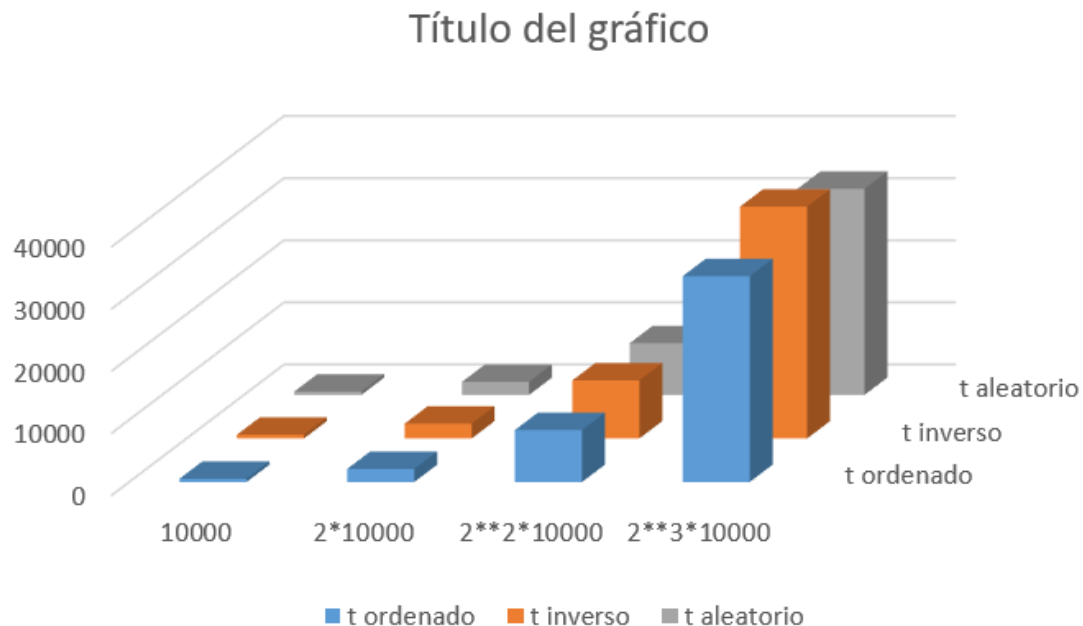
n	t ordenado	t inverso	t aleatorio
10000	529 ms = 0,529 s	594 ms = 0,594 s	537 ms = 0,537 s
2*10000	2110 ms = 2,11 s	2376 ms = 2,376 s	2100 ms = 2,1 s
2**2*10000	8369 ms = 8,369 s	9338 ms = 9,338 s	8324 ms = 8,324 s
2**3*10000	33120 ms = 33,12 s	37305 ms = 37,305 s	33167 ms = 33,167 s
2**4*10000	TFD >= 60 s	TFD >= 60 s	TFD >= 60 s

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal estudiada.

SOLUCIÓN:

Los resultados obtenidos siguen la misma linealidad que en el ejercicio anterior, una $O(n^2)$.

Sin embargo, en este caso, los tiempos de inversa y aleatorio son muy parecidos al ordenado, lo cual implica que haya poca diferencia de tiempos al ordenar los valores.



En este gráfico, apreciamos como el crecimiento de los diferentes tipos de ordenación es muy parecido, pese a que cuanto mayor sea el tamaño de números a ordenar, se empezará a notar cada vez más la diferencia en tiempos.

Ejercicio 3:

Implementar una clase InsercionTiempos.java que tras ser ejecutada sirva para rellenar la siguiente tabla:

SOLUCIÓN:

n	t ordenado	t inverso	t aleatorio
10000	FdF < 50 ms	802 ms = 0,802 s	404 ms = 0,404 s
2*10000	FdF < 50 ms	3225 ms = 3,225 s	1604 ms = 1,604 s
2**2*10000	FdF < 50 ms	12798 ms = 12,798 s	6566 ms = 6,566 s
2**3*10000	FdF < 50 ms	51083 ms = 51,083 s	25568 ms = 25,568 s
2**4*10000	FdF < 50 ms	TFD >= 60 s	TFD >= 60 s
2**5*10000	FdF < 50 ms	TFD >= 60 s	TFD >= 60 s
2**6*10000	FdF < 50 ms	TFD >= 60 s	TFD >= 60 s
2**7*10000	FdF < 50 ms	TFD >= 60 s	TFD >= 60 s
2**8*10000	71 ms = 0,071 s	TFD >= 60 s	TFD >= 60 s
2**9*10000	141 ms = 0,141 s	TFD >= 60 s	TFD >= 60 s
2**10*10000	281 ms = 0,281 s	TFD >= 60 s	TFD >= 60 s
2**11*10000	549 ms = 0,549 s	TFD >= 60 s	TFD >= 60 s
2**12*10000	1113 ms = 1,113 s	TFD >= 60 s	TFD >= 60 s
2**13*10000	2209 ms = 2,209 s	TFD >= 60 s	TFD >= 60 s

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal estudiada.

SOLUCIÓN:

En este ejercicio, tenemos una diferencia de complejidad muy notoria entre el ordenado, y el inverso y el aleatorio, ya que el ordenado sigue una complejidad $O(n)$ mientras que el resto, siguen una $O(n^2)$, como los vistos anteriormente.

Otra diferencia que se ve a simple vista es la gran diferencia en la medición de tiempos, ya que el ordenado consta de tiempos por debajo del segundo hasta tamaños muy grandes, mientras que el resto de las mediciones ya son superiores al minuto a partir de un tamaño donde el ordenado toma tiempos considerados poco fiables por ser menores de 50 ms

Ejercicio 4:

Implementar una clase RapidoTiempos.java que tras ser ejecutada sirva para rellenar la siguiente tabla:

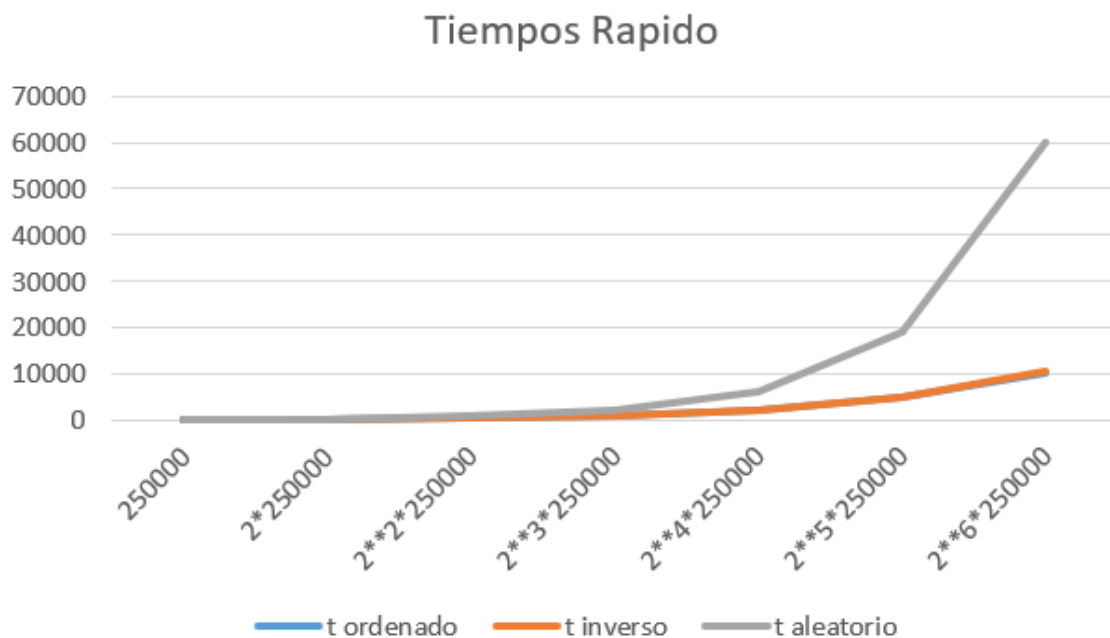
SOLUCIÓN:

n	t ordenado	t inverso	t aleatorio
250000	128 ms = 0,128 s	136 ms = 0,136 s	189 ms = 0,189 s
2*250000	257 ms = 0,257 s	274 ms = 0,274 s	423 ms = 0,423 s
2**2*250000	537 ms = 0,537 s	577 ms = 0,577 s	940 ms = 0,94 s
2**3*250000	1126 ms = 1,126 s	1220 ms = 1,22 s	2253 ms = 2,253 s
2**4*250000	2404 ms = 2,404 s	2493 ms = 2,493 s	6489 ms = 6,489 s
2**5*250000	4976 ms = 4,976 s	5175 ms = 5,175 s	18980 ms = 18,98 s
2**6*250000	10231 ms = 10,231 s	10728 ms = 10,728 s	TFD >= 60 s

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal estudiada.

SOLUCIÓN:

En los tiempos obtenidos se sigue una complejidad $O(n \log(n))$, por lo que los valores deben ser un poco superiores al doble, y se aprecia bien que es así.



Pese a que siguen la misma complejidad, y además es casi idéntica el de ordenado y el inverso, el aleatorio se dispara bastante, esto se puede deber a la aleatoriedad de la creación de números para su ordenación posterior.

ras ver lo que tarda el rápido en ordenar 16 millones de elementos inicialmente en orden aleatorio, calcule (a partir de las complejidades y los datos de las tablas anteriores), ¿cuántos días tardaría cada uno de esos tres métodos (Burbuja, Selección e Inserción) en hacer lo mismo?

SOLUCIÓN:

Sabemos que en un día hay 86.400 segundos. Usaremos una fórmula para calcular los tiempos. Usaremos un $n = 80.000$ y un $t_1 = 75$ segundos. Ahora cogemos el tamaño que buscamos y lo dividimos entre nuestro n . $k = 16.000.000/80.000 = 200$. Por último, calculamos el tiempo final. $t_2 = 200^2 * 75 \text{ s} = 3.000.000$ de segundos, que es el equivalente a 34.72 días.

Ejercicio 5:

Re implementar el método de inserción, para que en lugar de ordenar el vector que se le pasa en todas sus posiciones $[0 \dots longitud-1]$, lo ordene solo entre las posiciones [inferior .. superior], posiciones que se pasarán como parámetros.

Implementar una clase RapidoInsercion.java que ordene los elementos del vector según lo apuntado antes.

Ahora se propone hacer una toma de tiempos, que, aunque no sea concluyente y determinante para compararlos, sí será orientativa. Para ello, implementar una clase RapidoInsercionTiempos.java que tomará tiempos (para un tamaño de $n=16$ millones de elementos generados inicialmente de forma aleatoria) para rellenar la siguiente tabla:

SOLUCIÓN:

Caso	t aleatorio
Rapido	TFD $\geq 60 \text{ s}$
Rapido + Insercion ($k=5$)	TFD $\geq 60 \text{ s}$
Rapido + Insercion ($k=10$)	TFD $\geq 60 \text{ s}$
Rapido + Insercion ($k=20$)	TFD $\geq 60 \text{ s}$
Rapido + Insercion ($k=30$)	TFD $\geq 60 \text{ s}$
Rapido + Insercion ($k=50$)	TFD $\geq 60 \text{ s}$
Rapido + Insercion ($k=100$)	43354 ms = 43,354 s
Rapido + Insercion ($k=200$)	10899 ms = 10,899 s
Rapido + Insercion ($k=500$)	24137 ms = 24,137 s
Rapido + Insercion ($k=1000$)	43658 ms = 43,658 s

Razone conclusiones obtenidas a partir de la tabla anterior.

SOLUCIÓN:

Cuando trabajamos con tamaños tan grandes como 16.000.000, usando el método rápido y aleatorio, obtenemos un tiempo cercano a 60 segundos, pero mayor q este, por lo tanto, se encuentra fuera de nuestros límites. Aplicando el nuevo método, los resultados son los mismos para valores de k bajos, ya que con estos valores no apreciamos casi diferencia a ese valor inferior y superior. Sin embargo, cuando aumentamos el valor de k, obtenemos valores que, si se encuentran dentro de lo que buscamos, haciendo una forma de parábola.

